



ARISTOTLE
UNIVERSITY
OF THESSALONIKI

Βαθιά Μάθηση και Ανάλυση Πολυμεσικών Δεδομένων
Βαθιά Αναδρομικά Νευρωνικά Δίκτυα

Γεώργιος Κολοκοτρώνης

AM:143

Ημερομηνία: 31/05/2025

Εισαγωγή

Το IMDb dataset αποτελεί ένα από τα πιο διαδεδομένα σύνολα δεδομένων για την ανάλυση συναισθήματος, περιλαμβάνοντας 50.000 κριτικές, οι οποίες είναι μεταξύ θετικών και αρνητικών σχολίων. Το πρόβλημα ταξινόμησης σε θετικό/αρνητικό είναι ιδιαίτερα απαιτητικό λόγω της ποικιλίας και της πολυπλοκότητας της γλώσσας που χρησιμοποιούν οι χρήστες σε πραγματικές συνθήκες. Για την επίλυση αυτού του προβλήματος, επιλέχθηκε η χρήση του GRU, ενός τύπου αναδρομικού νευρωνικού δικτύου (RNN) που έχει αποδειχθεί ικανό να μοντελοποιεί σειριακά δεδομένα. Μετά την εκπαίδευση, πραγματοποιείται η τελική αξιολόγηση στο test set, το οποίο περιέχει δείγματα που το μοντέλο δεν έχει δει κατά τη διάρκεια της εκπαίδευσης. Για την ποιοτική αξιολόγηση του μοντέλου, εκτυπώνονται ορισμένα παραδείγματα σωστής και λανθασμένης κατηγοριοποίησης. Επιπλέον, υπολογίζονται βασικοί δείκτες απόδοσης, όπως η ακρίβεια (accuracy) και ο F1-score, προκειμένου να εκτιμηθεί η συνολική αποτελεσματικότητα του μοντέλου στο πρόβλημα της δυαδικής ταξινόμησης. Η χρήση του confusion matrix επιτρέπει την ανάλυση των τύπων σφαλμάτων (false positives, false negatives), παρέχοντας μια πιο ολοκληρωμένη εικόνα των δυνατοτήτων και αδυναμιών του μοντέλου. Στο τέλος έγινε και η δοκιμή ενός RNN LSTM.

Μεθοδολογία

Βασικές βιβλιοθήκες:

PyTorch (torch, nn, optim, utils.data), Transformers (BertTokenizer), Pandas, NumPy, scikit-learn (train_test_split), Matplotlib (plt), time.

```
# Check for GPU
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")
```

Ορίζουμε τη συσκευή εκτέλεσης (CPU ή GPU) χρησιμοποιώντας torch.device, για να επιταχύνουμε την εκπαίδευση του μοντέλου αν υπάρχει διαθέσιμη GPU. (Που υπάρχει στην περίπτωση μας: NVIDIA GeForce RTX 4050 Laptop GPU)

Διαχείριση Δεδομένων και Προεπεξεργασία:

```
dataset_path = r"C:\Users\kgoel\Downloads\archive\IMDBDataset.csv"
df = pd.read_csv(dataset_path)
df['sentiment'] = df['sentiment'].map({'positive': 1, 'negative': 0})
```

```
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
max_len = 256
```

Φορτώνουμε το IMDb dataset (50.000 κριτικές) από αρχείο CSV και μετατρέπουμε τις ετικέτες σε δυαδική μορφή (positive=1, negative=0) για ταξινόμηση συναισθήματος. Χρησιμοποιούμε τον προεκπαιδευμένο BERT tokenizer (bert-base-uncased) για να μετατρέψουμε τις κριτικές σε token IDs, κατάλληλα για είσοδο σε νευρωνικό δίκτυο, με μέγιστο μήκος ακολουθίας 256 tokens.

Dataset κλάση (IMDBDataset):

```
class IMDBDataset(Dataset):
    def __init__(self, texts, labels, tokenizer, max_len):
        self.texts = texts
        self.labels = labels
        self.tokenizer = tokenizer
        self.max_len = max_len

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, idx):
        text = self.texts[idx]
        label = self.labels[idx]
        tokens = self.tokenizer.encode(text, truncation=True, padding="max_length", max_length=self.max_len)
        return torch.tensor(label, dtype=torch.float32), torch.tensor(tokens)

texts = df['review'].tolist()
labels = df['sentiment'].tolist()
```

Ορίζουμε την κλάση `IMDBDataset` που παίρνει από το `PyTorch Dataset`. Αυτή η κλάση λαμβάνει ως είσοδο τις κριτικές (texts), τις ετικέτες (labels), τον tokenizer και το μέγιστο μήκος ακολουθίας. Στην `getitem`, κάθε κριτική γίνεται token με χρήση του BERT tokenizer, ενώ οι ετικέτες μετατρέπονται σε tensors κατάλληλα για εκπαίδευση σε PyTorch μοντέλο.

Χωρισμός Δεδομένων και DataLoaders:

```
X_train, X_temp, y_train, y_temp = train_test_split(texts, labels, test_size=0.3, random_state=42)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.5, random_state=42)

train_dataset = IMDBDataset(X_train, y_train, tokenizer, max_len)
val_dataset = IMDBDataset(X_val, y_val, tokenizer, max_len)
test_dataset = IMDBDataset(X_test, y_test, tokenizer, max_len)

train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=32)
test_loader = DataLoader(test_dataset, batch_size=32)
```

Τα δεδομένα χωρίζονται σε τρία σύνολα: train (70%), val(15%) και test(15%). Στη συνέχεια, δημιουργούνται τα αντίστοιχα datasets (train, validation, test) χρησιμοποιώντας την `IMDBDataset` κλάση, όπου κάθε δείγμα περιέχει το tokenized κείμενο και την ετικέτα. Τέλος, χρησιμοποιείται ο `DataLoader` της PyTorch για να διαχειριστεί τα δεδομένα σε batches (batch_size=32) και να τα ανακατεύει στην εκπαίδευση (shuffle=True).

Ορισμός GRU Μοντέλου

```
class RNNModel(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, output_dim, num_layers=2, dropout=0.5):
        super(RNNModel, self).__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.rnn = nn.GRU(embed_dim, hidden_dim, num_layers=num_layers, dropout=dropout, batch_first=True)
        self.fc = nn.Linear(hidden_dim, output_dim)
        self.dropout = nn.Dropout(dropout)

    def forward(self, text):
        embedded = self.embedding(text)
        output, hidden = self.rnn(embedded)
        hidden = self.dropout(hidden[-1])
        return self.fc(hidden)

vocab_size = tokenizer.vocab_size
embed_dim = 128
hidden_dim = 256
output_dim = 1
num_layers = 2

model = RNNModel(vocab_size, embed_dim, hidden_dim, output_dim, num_layers).to(device)
```

Το μοντέλο `RNNModel` υλοποιεί ένα επαναληπτικό νευρωνικό δίκτυο (RNN) βασισμένο στην αρχιτεκτονική GRU (Gated Recurrent Unit). Περιλαμβάνει:

- Embedding layer για τη μετατροπή των token IDs σε διανύσματα πυκνής αναπαράστασης (word embeddings).
- Ένα GRU με δύο επαναληπτικά στρώματα (num_layers=2), το οποίο επεξεργάζεται τις ακολουθίες λέξεων και μαθαίνει χρονικές εξαρτήσεις.

- Ένα πλήρως συνδεδεμένο (linear) layer που δίνει την έξοδο της ταξινόμησης (θετικό/αρνητικό συναίσθημα).
- Dropout layer για την αποφυγή υπερπροσαρμογής (overfitting).

Η κλάση υλοποιεί τη μέθοδο forward, η οποία καθορίζει τη ροή των δεδομένων μέσα στο μοντέλο.

Loss Function, Optimizer και Early Stopping

```
criterion = nn.BCEWithLogitsLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

class EarlyStopping:
    def __init__(self, patience=5, verbose=True, delta=0):
        self.patience = patience
        self.verbose = verbose
        self.counter = 0
        self.best_score = None
        self.early_stop = False
        self.val_loss_min = np.Inf
        self.delta = delta

    def __call__(self, val_loss, model):
        score = -val_loss
        if self.best_score is None:
            self.best_score = score
            self.save_checkpoint(val_loss, model)
        elif score < self.best_score + self.delta:
            self.counter += 1
            if self.verbose:
                print(f"EarlyStopping counter: {self.counter} out of {self.patience}")
            if self.counter >= self.patience:
                self.early_stop = True
        else:
            self.best_score = score
            self.save_checkpoint(val_loss, model)
            self.counter = 0

    def save_checkpoint(self, val_loss, model):
        print(f"Validation loss decreased ({self.val_loss_min:.6f} --> {val_loss:.6f}). Saving model ...")
        torch.save(model.state_dict(), "checkpoint.pth")
        self.val_loss_min = val_loss

early_stopping = EarlyStopping(patience=5, verbose=True)
```

Για την εκπαίδευση του μοντέλου χρησιμοποιείται η συνάρτηση απώλειας BCEWithLogitsLoss (Binary Cross-Entropy with Logits), η οποία συνδυάζει την απώλεια binary cross-entropy με το sigmoid activation. Αυτή η συνάρτηση είναι κατάλληλη για προβλήματα δυαδικής ταξινόμησης, όπως η ανάλυση συναισθήματος (θετικό/αρνητικό). Ο optimizer που επιλέγεται είναι ο Adam, ένας προσαρμοζόμενος αλγόριθμος βελτιστοποίησης πολύ κλασικός για τασκς, προσφέροντας γρήγορη και σταθερή σύγκλιση. Το learning rate ορίζεται σε 0.001. Για την αποφυγή υπερπροσαρμογής, υλοποιείται μηχανισμός EarlyStopping: παρακολουθεί την τιμή του validation loss και σταματά την εκπαίδευση αν το μοντέλο δεν παρουσιάζει βελτίωση για πέντε συνεχόμενες εποχές με patience=5.

Εκπαίδευση Μοντέλου

```
def train_model(model, train_loader, val_loader, criterion, optimizer, num_epochs, early_stopping):
    train_losses = []
    val_losses = []
    for epoch in range(num_epochs):
        start_time = time.time()
        model.train()
        running_loss = 0.0
        for labels, texts in train_loader:
            labels, texts = labels.to(device), texts.to(device)
            optimizer.zero_grad()
            outputs = model(texts)
            loss = criterion(outputs.squeeze(), labels)
            loss.backward()
            optimizer.step()
            running_loss += loss.item()
        train_loss = running_loss / len(train_loader)
        train_losses.append(train_loss)

        model.eval()
        val_loss = 0.0
        correct = 0
        total = 0
        with torch.no_grad():
            for labels, texts in val_loader:
                labels, texts = labels.to(device), texts.to(device)
                outputs = model(texts)
                loss = criterion(outputs.squeeze(), labels)
                val_loss += loss.item()
                preds = torch.sigmoid(outputs).cpu().numpy()
                preds = (preds >= 0.5).astype(int)
                correct += (preds.flatten() == labels.cpu().numpy()).sum()
                total += labels.size(0)
        val_loss = val_loss / len(val_loader)
        val_losses.append(val_loss)
        val_acc = 100 * correct / total
        epoch_time = time.time() - start_time
        print(f"Epoch {epoch+1}, Train Loss: {train_loss:.4f}, Val Loss: {val_loss:.4f}, Val Acc: {val_acc:.2f}%, Time: {epoch_time:.2f}s")

        early_stopping(val_loss, model)
        if early_stopping.early_stop:
            print("Early stopping")
            break
    return train_losses, val_losses

EPOCHS = 15
train_losses, val_losses = train_model(model, train_loader, val_loader, criterion, optimizer, EPOCHS, early_stopping)
```

Η εκπαίδευση του μοντέλου πραγματοποιείται μέσω της συνάρτησης `train_model`, η οποία εκτελείται για προκαθορισμένο αριθμό εποχών (`num_epochs=15`). Για κάθε εποχή, το μοντέλο εκπαιδεύεται σε batches από το training set, υπολογίζοντας το loss και εκτελώντας βήματα backward propagation και βελτιστοποίησης μέσω του optimizer. Μετά την εκπαίδευση σε κάθε εποχή, αξιολογείται η απόδοση του μοντέλου στο validation set, υπολογίζοντας το μέσο validation loss και την ακρίβεια (accuracy). Ο μηχανισμός EarlyStopping παρακολουθεί το validation loss και σταματά την εκπαίδευση εάν δεν παρατηρηθεί βελτίωση για 5 συνεχόμενες εποχές. Η συνάρτηση επιστρέφει τις λίστες με τις τιμές του training loss και του validation loss ανά εποχή, ώστε να μπορούν να απεικονιστούν σε γραφήματα.

Οπτικοποίηση Εκπαίδευσης

```
epochs_range = range(1, len(train_losses) + 1)
plt.figure(figsize=(10, 5))
plt.plot(epochs_range, train_losses, marker="o", linestyle="-", label="Training Loss")
plt.plot(epochs_range, val_losses, marker="o", linestyle="-", label="Validation Loss")
plt.title("Training and Validation Loss per Epoch")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)
plt.show()
```

Μετά την εκπαίδευση, δημιουργείται ένα γράφημα για την παρακολούθηση της πορείας του training loss και του validation loss σε κάθε εποχή. Το γράφημα απεικονίζει την εξέλιξη των τιμών αυτών των δεικτών, επιτρέποντας τον εντοπισμό περιπτώσεων υπερπροσαρμογής (overfitting) ή υποπροσαρμογής (underfitting).

Αξιολόγηση Μοντέλου στο Test Set

```
def evaluate_model(model, test_loader):
    model.eval()
    correct = 0
    total = 0
    all_preds = []
    all_labels = []
    with torch.no_grad():
        for labels, texts in test_loader:
            labels, texts = labels.to(device), texts.to(device)
            outputs = model(texts)
            preds = torch.sigmoid(outputs).cpu().numpy()
            preds = (preds >= 0.5).astype(int)
            correct += (preds.flatten() == labels.cpu().numpy()).sum()
            total += labels.size(0)
            all_preds.extend(preds.flatten())
            all_labels.extend(labels.cpu().numpy())
    accuracy = 100 * correct / total
    print(f"Test Accuracy: {accuracy:.2f}%")
    return all_preds, all_labels

model.load_state_dict(torch.load("checkpoint.pth"))
test_preds, test_labels = evaluate_model(model, test_loader)
```

Μετά την εκπαίδευση, το καλύτερο μοντέλο (αποθηκευμένο μέσω του μηχανισμού EarlyStopping) φορτώνεται από το αρχείο checkpoint.pth. Στη συνέχεια, γίνεται η τελική αξιολόγηση στο test set, το οποίο περιέχει δείγματα που δεν έχουν χρησιμοποιηθεί κατά την εκπαίδευση ή την επικύρωση.

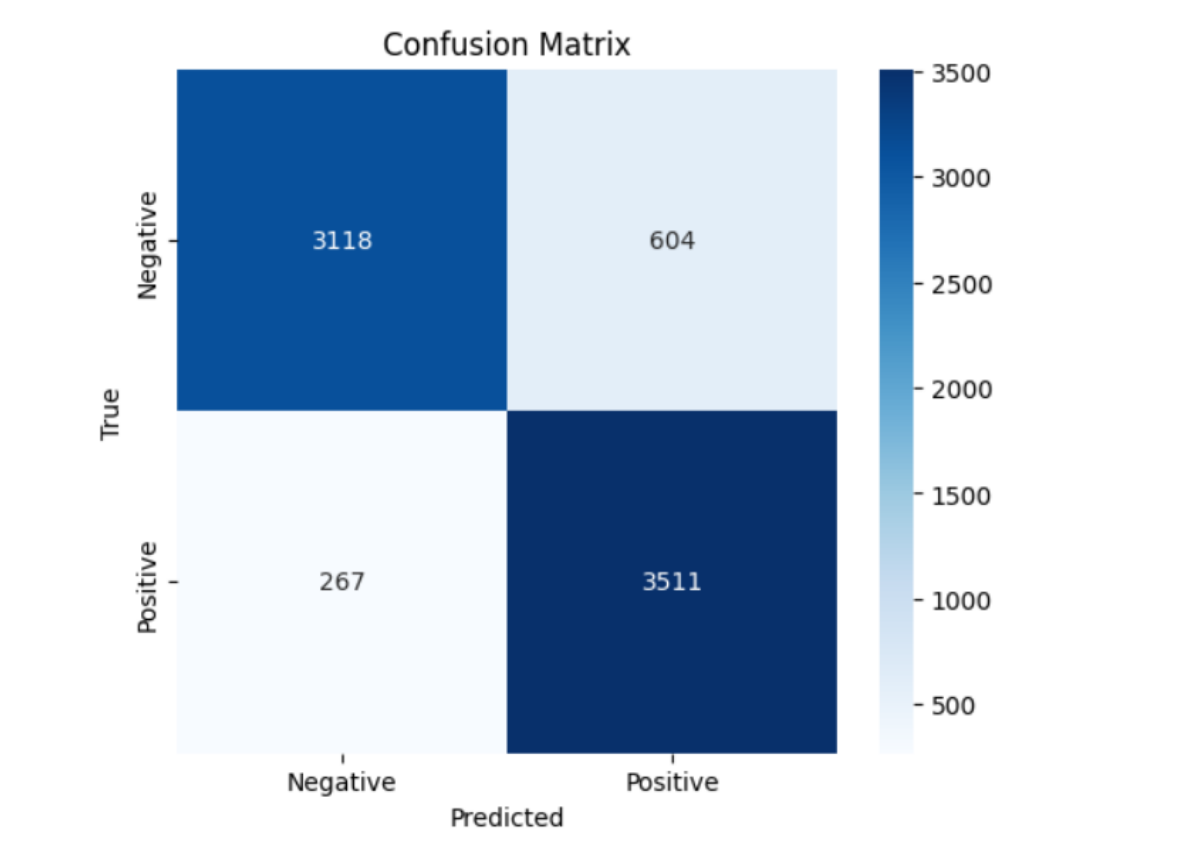
Η συνάρτηση evaluate_model υπολογίζει την ακρίβεια του μοντέλου (Test Accuracy) συγκρίνοντας τις προβλέψεις (με κατώφλι 0.5) με τις πραγματικές ετικέτες, ενώ επιστρέφει και τις πλήρεις λίστες με τις προβλέψεις (test_preds) και τις πραγματικές ετικέτες (test_labels) για περαιτέρω ανάλυση.

Αποτελέσματα εκπαίδευσης(GRU)

```
Using device: cuda
Epoch 1, Train Loss: 0.5828, Val Loss: 0.3655, Val Acc: 84.25%, Time: 312.44s
Validation loss decreased (inf --> 0.365482). Saving model ...
Epoch 2, Train Loss: 0.2921, Val Loss: 0.2731, Val Acc: 88.35%, Time: 223.90s
Validation loss decreased (0.365482 --> 0.273121). Saving model ...
Epoch 3, Train Loss: 0.1951, Val Loss: 0.2898, Val Acc: 88.47%, Time: 231.79s
EarlyStopping counter: 1 out of 5
Epoch 4, Train Loss: 0.1141, Val Loss: 0.3392, Val Acc: 88.08%, Time: 233.10s
EarlyStopping counter: 2 out of 5
Epoch 5, Train Loss: 0.0609, Val Loss: 0.4441, Val Acc: 88.05%, Time: 226.01s
EarlyStopping counter: 3 out of 5
Epoch 6, Train Loss: 0.0394, Val Loss: 0.5753, Val Acc: 86.37%, Time: 226.46s
EarlyStopping counter: 4 out of 5
Epoch 7, Train Loss: 0.0306, Val Loss: 0.4893, Val Acc: 87.60%, Time: 236.00s
EarlyStopping counter: 5 out of 5
Early stopping
```

Το μοντέλο GRU παρουσίασε ιδιαίτερα καλή συμπεριφορά κατά την εκπαίδευση στο IMDb dataset. Στις πρώτες εποχές, παρατηρήθηκε σημαντική μείωση τόσο του training loss όσο και του validation loss, υποδεικνύοντας ότι το μοντέλο μάθαινε αποτελεσματικά να ξεχωρίζει τα θετικά από τα αρνητικά συναισθήματα στις κριτικές ταινιών. Η ακρίβεια στο validation set έφτασε από 84.25% στην πρώτη εποχή σε 88.35% στη δεύτερη, δείχνοντας γρήγορη βελτίωση. Στη συνέχεια, το training loss συνέχισε να μειώνεται σταθερά, φτάνοντας σε πολύ χαμηλές τιμές (0.0609 στην 5η εποχή και 0.0306 στην 7η), ενώ το validation loss εμφάνισε μια χαρακτηριστική καμπύλη overfitting: σταμάτησε να βελτιώνεται μετά τη δεύτερη εποχή και άρχισε να αυξάνεται σταδιακά, φτάνοντας τα 0.5753 στην 6η εποχή. Αυτό είναι ένα συνηθισμένο φαινόμενο, καθώς το μοντέλο προσαρμόζεται υπερβολικά στα δεδομένα εκπαίδευσης, χάνοντας την ικανότητα γενίκευσης σε νέα δεδομένα. Παρόλα αυτά, η ακρίβεια στο validation set παρέμεινε σταθερά υψηλή (~88%) για αρκετές εποχές, γεγονός που υποδηλώνει ότι το μοντέλο ήταν αρκετά στιβαρό παρά την έναρξη του overfitting. Ο μηχανισμός EarlyStopping λειτούργησε σωστά, σταματώντας την εκπαίδευση στην 7η εποχή, αφού το validation loss δεν είχε βελτιωθεί για πέντε συνεχόμενες εποχές. Το καλύτερο μοντέλο αποθηκεύτηκε από την 2η εποχή, εξασφαλίζοντας τη χρήση της βέλτιστης έκδοσης του δικτύου για την τελική αξιολόγηση. Συνολικά, το μοντέλο κατάφερε να επιτύχει υψηλή ακρίβεια στην ανάλυση συναισθήματος, δείχνοντας την ικανότητα του GRU να κατανοεί τις χρονικές σχέσεις και τις εξαρτήσεις στο κείμενο, ενώ η χρήση του validation set και του early stopping προστάτευσαν από φαινόμενα υπερπροσαρμογής. Η ακρίβεια του test set 88.69%!

Confusion Matrix



3 Σωστές και 3 Λανθασμένες Κατηγοριοποιήσεις

3 Σωστές Προβλέψεις:

Review: Eleven "great" filmmakers, eleven pieces of garbage. Eleven minutes each of sheer tedium, sophistry, condescension, self-indulgence. Treats for people of all nations. Yussef Chahine of Egypt giving a "hip hip hooray!" for terrorism in his amateurish segment. Across the green line we have Amos Gitai o...
Prediction: 0, Label: 0.0

Review: I was intrigued by the nasty boss character as I am one myself, and the actual boss's daughter was attractive and it was interesting to see an even younger Ashton Kushner, but this movie is so puerile I had to turn it off. It was a waste of time to watch it. When people started peeing all over the l...
Prediction: 0, Label: 0.0

Review: "Problem Child" is one of the goofiest movies ever made. It's not the worst (though some people will disagree with me on that), but it's not the best either. It's about a devilish 7-year-old boy who wrecks comic havoc on a childless couple (John Ritter, Amy Yasbeck) who foolishly adopts him. This fi...
Prediction: 0, Label: 0.0

3 Λάθος Προβλέψεις:

Review: The biggest National Lampoon hit remains "Animal House", and rightly so. It was funny, raucous and good-natured.

The exact opposite of every other National Lampoon film. Including "Class Reunion".

PLEASE do not be fooled by the inclusion of Stephen Furst ("Flounder") from "Animal House".
Prediction: 1, Label: 0.0

Review: It is a shame that a movie with such a good cinematography as this one had no plot to be supported by the work of Sarah Cawley (cinematography) and Adam Lichtenstein (Film Editing), and above all, no sense of what goes on in Mexico City. The movie tries to be a very realistic depiction of life in ci...
Prediction: 1, Label: 0.0

Review: It was great to see some of my favorite stars of 30 years ago including John Ritter, Ben Gazzara and Audrey Hepburn. They looked quite wonderful. But that was it. They were not given any characters or good lines to work with. I neither understood or cared what the characters were doing.

S...
Prediction: 1, Label: 0.0

Περιγραφή Classification Report

Classification Report:				
	precision	recall	f1-score	support
Negative	0.92	0.84	0.88	3722
Positive	0.85	0.93	0.89	3778
accuracy			0.88	7500
macro avg	0.89	0.88	0.88	7500
weighted avg	0.89	0.88	0.88	7500

Το μοντέλο GRU πέτυχε πολύ καλή απόδοση στην ταξινόμηση στο IMDB dataset, με συνολική ακρίβεια 88% στο test set. Αναλυτικότερα, η ακρίβεια για τις αρνητικές κριτικές ήταν υψηλή (precision 0.92), με recall 0.84 και F1-score 0.88, ενώ για τις θετικές κριτικές παρατηρήθηκε υψηλό recall (0.93) και ελαφρώς χαμηλότερο precision (0.85), με F1-score 0.89. Οι τιμές macro και weighted F1-score ήταν 0.88, επιβεβαιώνοντας τη συνολικά ισορροπημένη απόδοση του μοντέλου σε ένα dataset με σχεδόν ίσο αριθμό θετικών και αρνητικών δειγμάτων. Το μοντέλο καταφέρνει να ανιχνεύει σωστά την πλειοψηφία των συναισθηματικών τάσεων στα κείμενα, με λίγα λάθη κυρίως στην ταξινόμηση θετικών κριτικών ως αρνητικές.

LSTM

```
class RNNModel(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, output_dim, num_layers=2, dropout=0.5):
        super(RNNModel, self).__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.rnn = nn.GRU(embed_dim, hidden_dim, num_layers=num_layers, dropout=dropout, batch_first=True)
        self.fc = nn.Linear(hidden_dim, output_dim)
        self.dropout = nn.Dropout(dropout)

    def forward(self, text):
        embedded = self.embedding(text)
        output, hidden = self.rnn(embedded)
        hidden = self.dropout(hidden[-1])
        return self.fc(hidden)

vocab_size = tokenizer.vocab_size
embed_dim = 128
hidden_dim = 256
output_dim = 1
num_layers = 2

model = RNNModel(vocab_size, embed_dim, hidden_dim, output_dim, num_layers).to(device)

criterion = nn.BCEWithLogitsLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

Μετά από το μοντέλο GRU, πραγματοποιήθηκε δοκιμή και με ένα δεύτερο μοντέλο βασισμένο στην αρχιτεκτονική LSTM (Long Short-Term Memory), το οποίο είναι μια δημοφιλής παραλλαγή των Recurrent Neural Networks. Ο κώδικας του LSTM μοντέλου είναι σχεδόν ίδιος με αυτόν του GRU, με μικρές διαφορές που αφορούν κυρίως στο εσωτερικό του layer: αντί για το nn.GRU

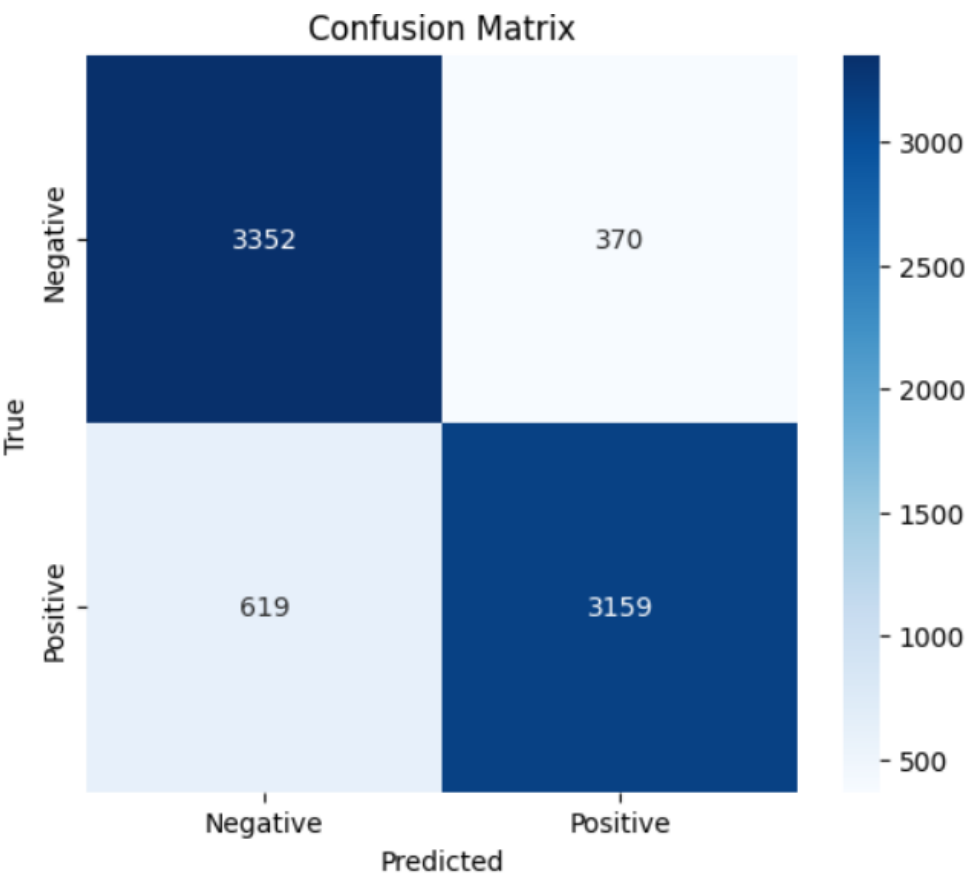
χρησιμοποιείται το nn.LSTM. Επίσης, στο forward pass, το LSTM επιστρέφει δύο στοιχεία, το hidden state και το cell state, σε αντίθεση με το GRU που επιστρέφει μόνο το hidden state. Αυτές οι αλλαγές είναι απαραίτητες λόγω της εσωτερικής δομής του LSTM, που διαθέτει επιπλέον πύλες και ξεχωριστή μνήμη (cell state), ώστε να διαχειρίζεται μακρινές εξαρτήσεις καλύτερα.

Αποτελέσματα εκπαίδευσης (LSTM)

```
Using device: cuda
Epoch 1, Train Loss: 0.6943, Val Loss: 0.6940, Val Acc: 49.19%, Time: 226.38s
Validation loss decreased (inf --> 0.693995). Saving model ...
Epoch 2, Train Loss: 0.6923, Val Loss: 0.6897, Val Acc: 55.01%, Time: 221.21s
Validation loss decreased (0.693995 --> 0.689709). Saving model ...
Epoch 3, Train Loss: 0.6644, Val Loss: 0.6946, Val Acc: 50.81%, Time: 224.56s
EarlyStopping counter: 1 out of 5
Epoch 4, Train Loss: 0.6457, Val Loss: 0.4992, Val Acc: 77.27%, Time: 219.12s
Validation loss decreased (0.689709 --> 0.499221). Saving model ...
Epoch 5, Train Loss: 0.4187, Val Loss: 0.3626, Val Acc: 82.97%, Time: 221.38s
Validation loss decreased (0.499221 --> 0.362619). Saving model ...
Epoch 6, Train Loss: 0.3041, Val Loss: 0.3201, Val Acc: 87.08%, Time: 218.97s
Validation loss decreased (0.362619 --> 0.320125). Saving model ...
Epoch 7, Train Loss: 0.2454, Val Loss: 0.3166, Val Acc: 86.83%, Time: 218.72s
Validation loss decreased (0.320125 --> 0.316555). Saving model ...
Epoch 8, Train Loss: 0.2023, Val Loss: 0.3590, Val Acc: 86.61%, Time: 218.64s
EarlyStopping counter: 1 out of 5
Epoch 9, Train Loss: 0.1615, Val Loss: 0.3723, Val Acc: 87.12%, Time: 219.65s
EarlyStopping counter: 2 out of 5
Epoch 10, Train Loss: 0.1204, Val Loss: 0.3561, Val Acc: 87.13%, Time: 219.05s
EarlyStopping counter: 3 out of 5
Epoch 11, Train Loss: 0.0940, Val Loss: 0.4351, Val Acc: 86.36%, Time: 222.95s
EarlyStopping counter: 4 out of 5
Epoch 12, Train Loss: 0.0704, Val Loss: 0.5069, Val Acc: 86.64%, Time: 223.44s
EarlyStopping counter: 5 out of 5
Early stopping
```

Το μοντέλο LSTM παρουσίασε ικανοποιητικά αποτελέσματα κατά την εκπαίδευση στο IMDB dataset. Στις πρώτες δύο εποχές, παρατηρήθηκε αργή μείωση του training και validation loss, με την ακρίβεια στο validation set να παραμένει γύρω στο 55%. Ωστόσο, από την 4η εποχή και μετά, το μοντέλο άρχισε να μαθαίνει πιο καλά, σημειώνοντας σημαντική βελτίωση: το validation loss έπεσε δραστικά από 0.6897 σε 0.4992, ενώ η ακρίβεια ανέβηκε στο 77%. Στις επόμενες εποχές, αυτό συνεχίστηκε, με το validation loss να φτάνει σε πολύ χαμηλά επίπεδα (0.3166 στην 7η εποχή) και την ακρίβεια να ξεπερνά το 87%. Στη συνέχεια, παρατηρήθηκε σταθεροποίηση και ελαφρά αύξηση του validation loss, ενώ η ακρίβεια παρέμεινε γύρω στο 86%-87%. Ο μηχανισμός EarlyStopping ενεργοποιήθηκε στην 12η εποχή, καθώς το validation loss δεν βελτιωνόταν περαιτέρω για πέντε συνεχόμενες εποχές. Συνολικά, το μοντέλο παρουσίασε υψηλή απόδοση και ισχυρή γενίκευση, επιτυγχάνοντας ακρίβεια που ξεπερνά το 87.10% στο validation set και 86.81% στο tset set.

Confusion matrix



3 Σωστές και 3 Λανθασμένες Κατηγοριοποιήσεις

3 Σωστές Προβλέψεις:

Review: Eleven "great" filmmakers, eleven pieces of garbage. Eleven minutes each of sheer tedium, sophistry, condescension, self-indulgence. Treats for people of all nations. Yussef Chahine of Egypt giving a "hip hip hooray!" for terrorism in his amateurish segment. Across the green line we have Amos Gitai o...
Prediction: 0, Label: 0.0

Review: I was intrigued by the nasty boss character as I am one myself, and the actual boss's daughter was attractive and it was interesting to see an even younger Ashton Kushner, but this movie is so puerile I had to turn it off. It was a waste of time to watch it. When people started peeing all over the l...
Prediction: 0, Label: 0.0

Review: "Problem Child" is one of the goofiest movies ever made. It's not the worst (though some people will disagree with me on that), but it's not the best either. It's about a devilish 7-year-old boy who wrecks comic havoc on a childless couple (John Ritter, Amy Yasbeck) who foolishly adopts him. This fi...
Prediction: 0, Label: 0.0

3 Λανθασμένες Προβλέψεις:

Review: The biggest National Lampoon hit remains "Animal House", and rightly so. It was funny, raucous and good-natured.

The exact opposite of every other National Lampoon film. Including "Class Reunion".

PLEASE do not be fooled by the inclusion of Stephen Furst ("Flounder") from "Animal House".
Prediction: 1, Label: 0.0

Review: It is a shame that a movie with such a good cinematography as this one had no plot to be supported by the work of Sarah Cawley (cinematography) and Adam Lichtenstein (Film Editing), and above all, no sense of what goes on in Mexico City. The movie tries to be a very realistic depiction of life in ci...
Prediction: 1, Label: 0.0

Review: From start to finish, I laughed real hard throughout the whole movie. It's amazing that "The Groove Tube" is possibly the granddaddy, yet raunchiest, of all comedic skit movies. This is the way I enjoy watching TV without being bored at flipping channels only to suffer from insomnia! For 73 minutes, ...
Prediction: 0, Label: 1.0

Περιγραφή Classification Report

Classification Report:				
	precision	recall	f1-score	support
Negative	0.84	0.90	0.87	3722
Positive	0.90	0.84	0.86	3778
accuracy			0.87	7500
macro avg	0.87	0.87	0.87	7500
weighted avg	0.87	0.87	0.87	7500

Παρατηρούμε ότι το μοντέλο αποδίδει καλά και στις δύο κατηγορίες (θετική και αρνητική κριτική), με ακρίβεια (accuracy) στο test set 87%. Συγκεκριμένα, το μοντέλο είχε ελαφρώς καλύτερη precision στις θετικές κριτικές (0.90) και καλύτερη recall στις αρνητικές κριτικές (0.90). Το f1-score είναι σχεδόν ισοδύναμο και για τις δύο κατηγορίες (0.87 και 0.86), υποδεικνύοντας καλή ισορροπία μεταξύ precision και recall. Συνολικά, το μοντέλο δείχνει ικανοποιητική απόδοση και καλή ικανότητα γενίκευσης στο test set.

